Context Parallelism / Dynamic CP

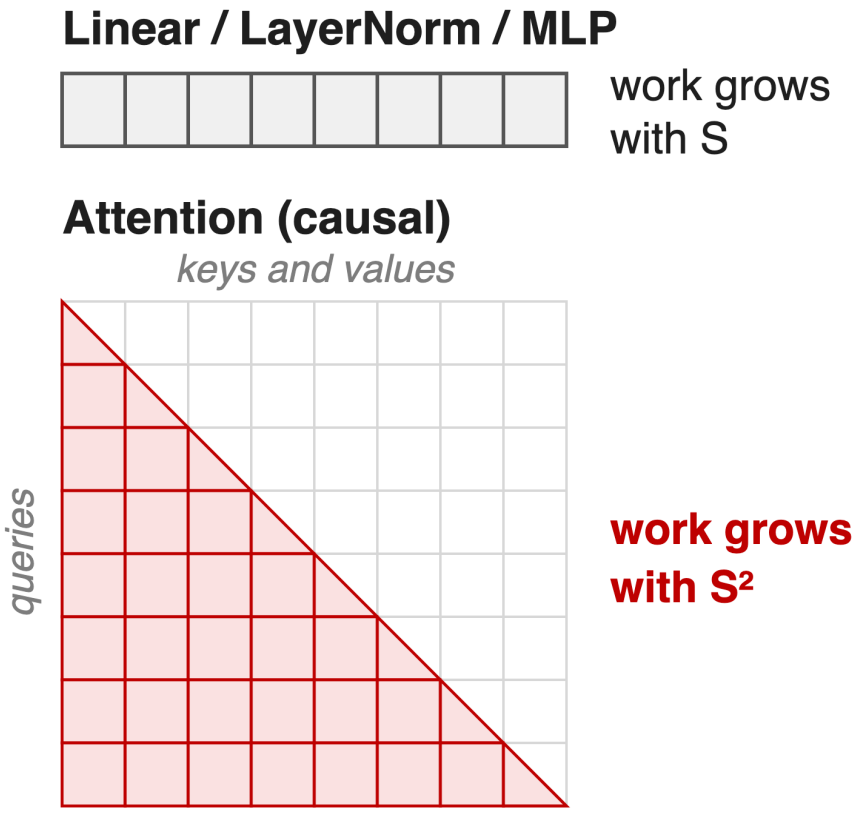
Shard the sequence; resize the shards per micro-batch

NVIDIA · Megatron-Core + Transformer Engine

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Long sequences make attention dominate

- Reasoning and RL runs: **16K–64K+** tokens
- At **64K**, attention is **69%** of FLOPs (10–15% when short)
- Activation memory grows ~linearly with sequence length S



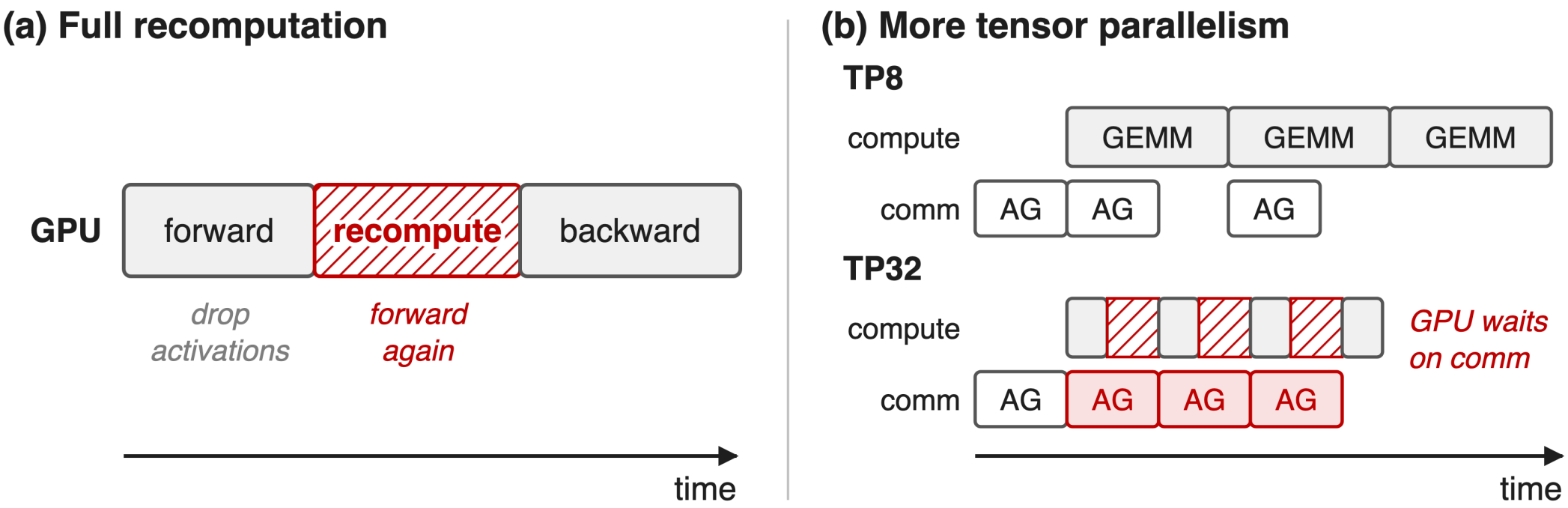
Recomputing to fit long context costs ~30%

~30%

extra training time from full activation recomputation, the default way to avoid OOM on long sequences

Megatron-LM user guide, Context Parallelism, "Context Parallelism Benefits"

Both memory knobs slow training down



Schematic, not to scale. Megatron sequence parallelism splits only LayerNorm and Dropout activations

More TP alone: 175B GPT at 32K drops to ≈0.85× of full recompute

Insight: only attention mixes tokens

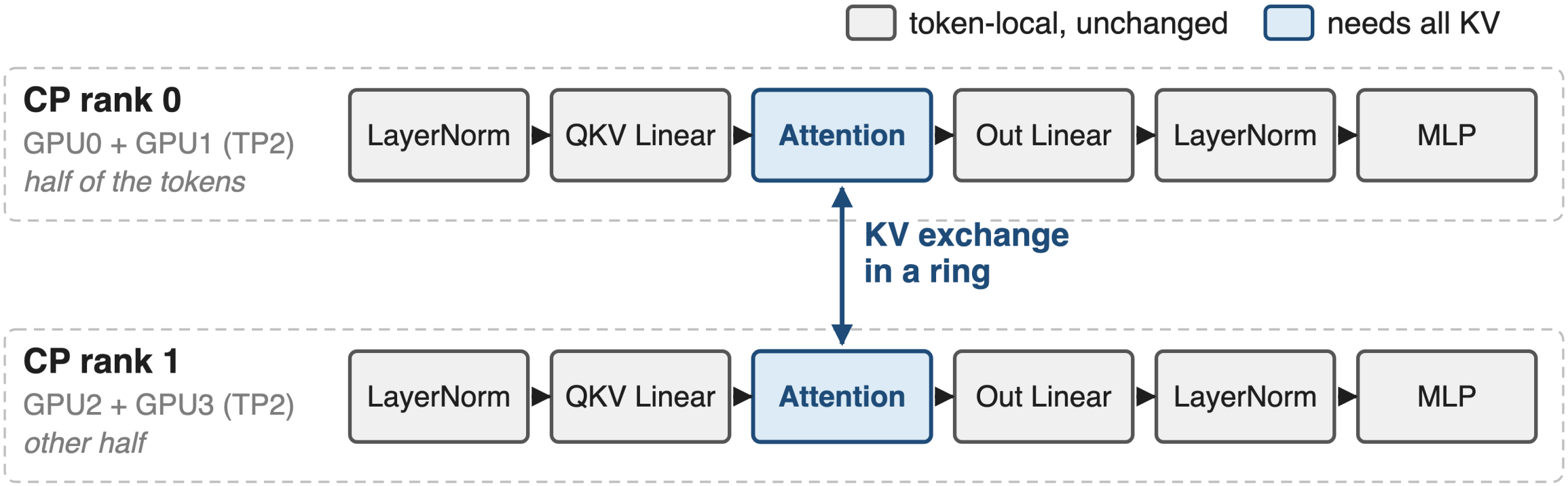
- Linear, LayerNorm, MLP: every token on its own
- Attention: each query needs earlier tokens' K and V

Insight: only attention mixes tokens

- Linear, LayerNorm, MLP: every token on its own
- Attention: each query needs earlier tokens' K and V

Split the sequence: everything stays local except KV, and KV can move while attention computes

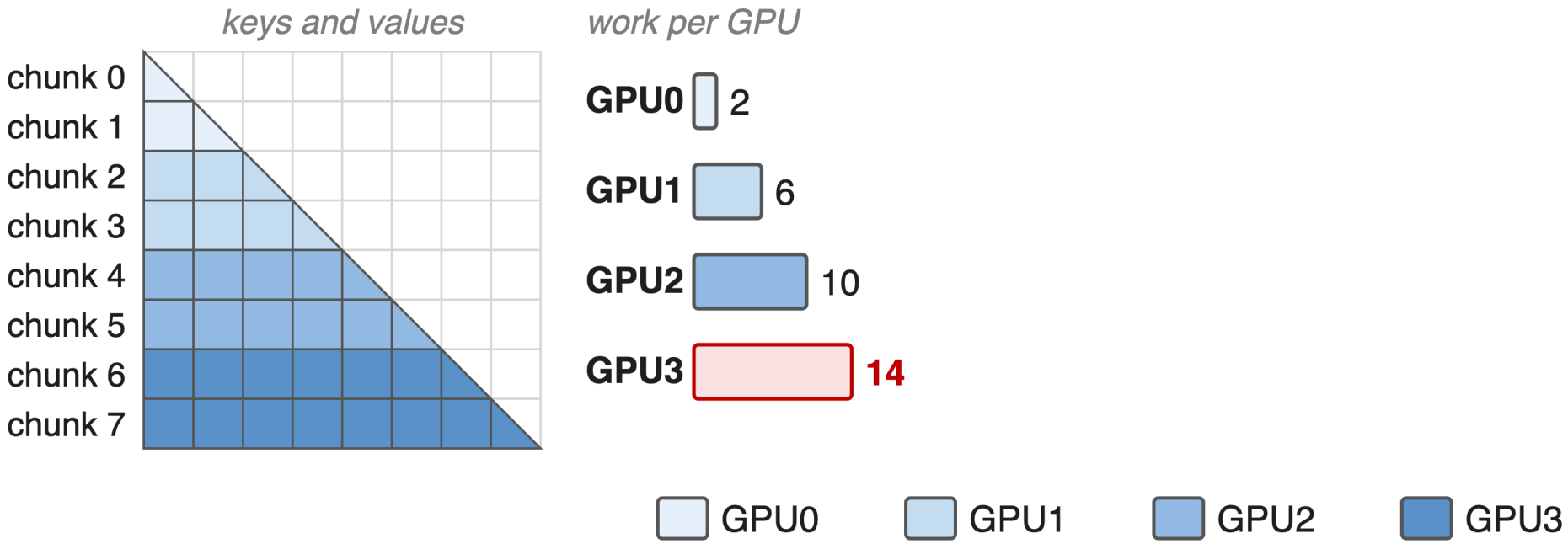
CP splits every activation along the sequence



World size = TP × CP × PP × DP; --context-parallel-size N (MCore ≥ 0.5, TE ≥ 1.1). GPU0↔GPU2 and GPU1↔GPU3 swap KV

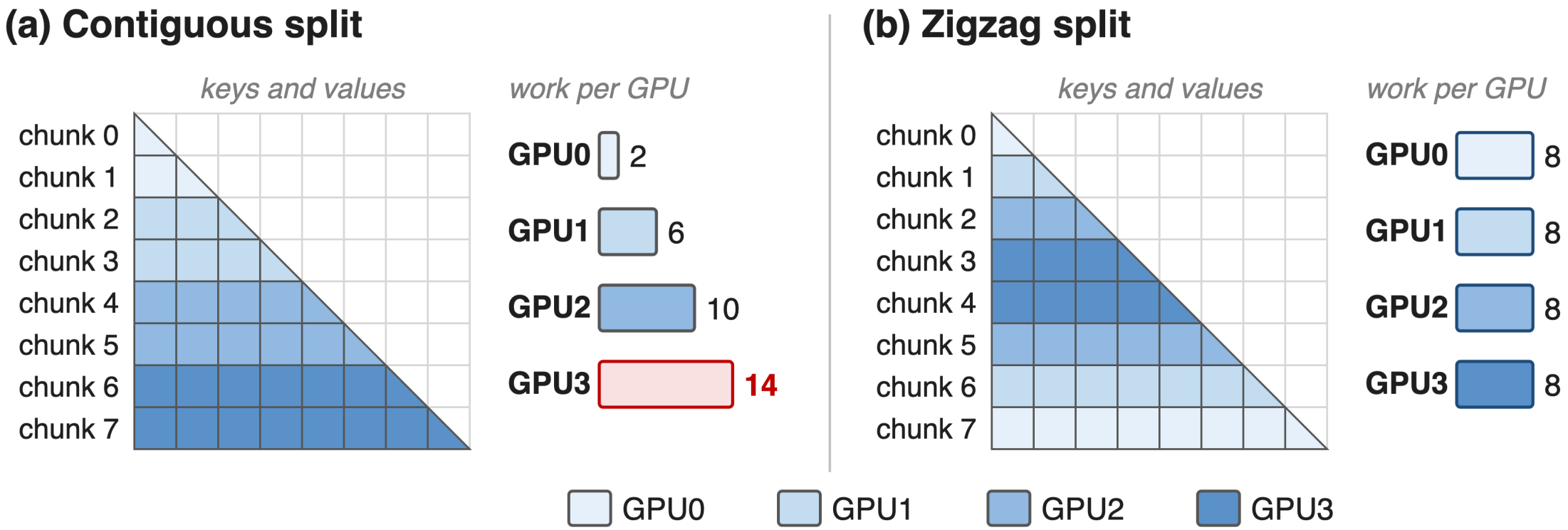
❶ Zigzag chunks balance the causal triangle

(a) Contiguous split



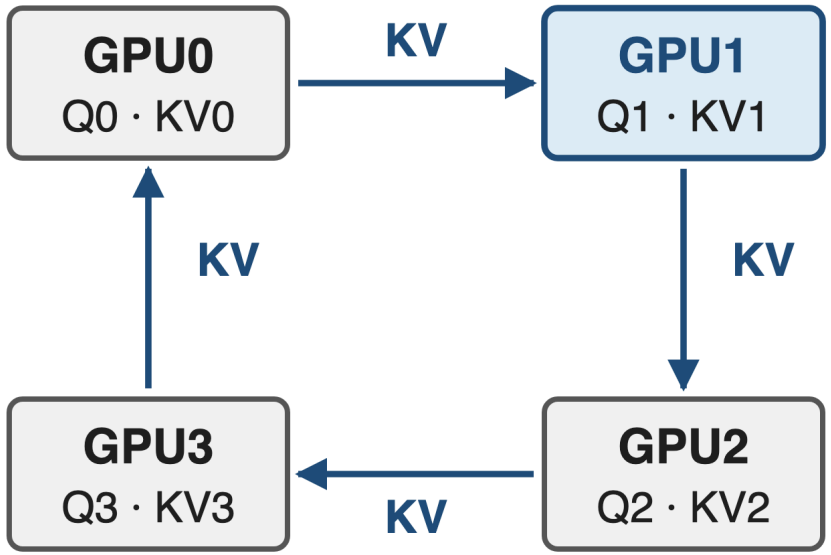
Work in chunk-block units (analytic). Sequence cut into 2 × CP chunks

❶ Zigzag chunks balance the causal triangle

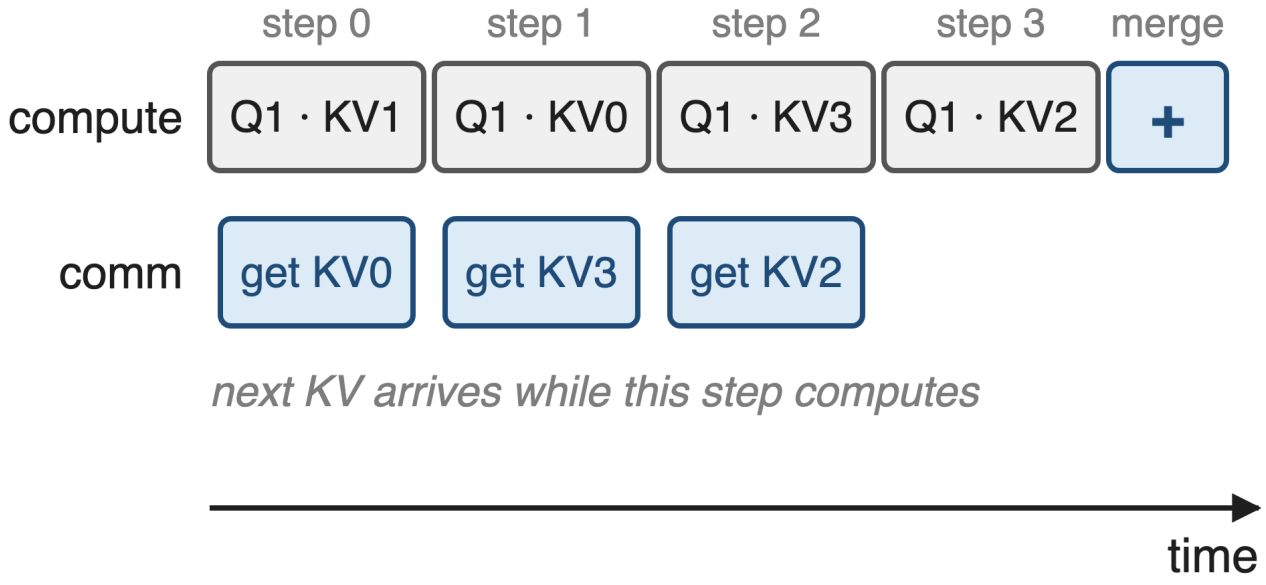


GPU r holds chunks r and 2CP−1−r, so every GPU does 8 units; sequence length must divide by 2 × CP

❷ KV circulates in a ring under attention



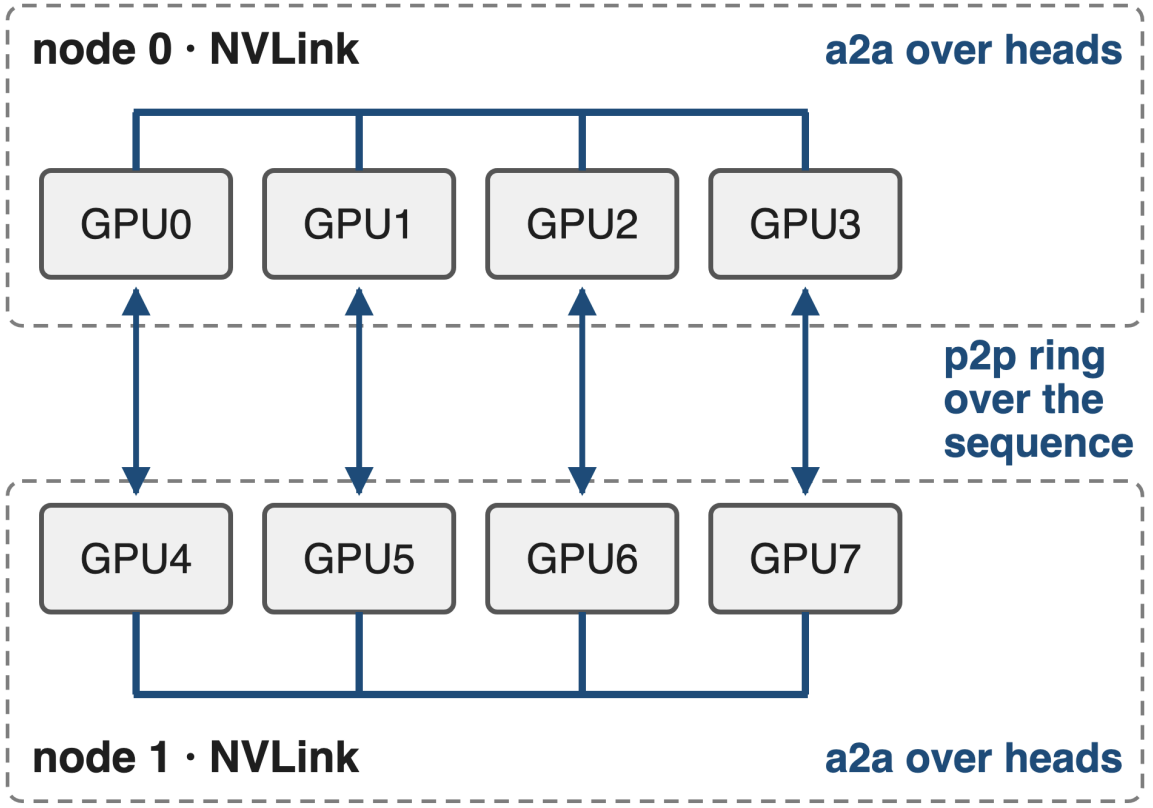
GPU1 timeline in one attention layer



Backward sends KV around again and returns dKV; only local KV is stored, and GQA/MQA shrink the bytes

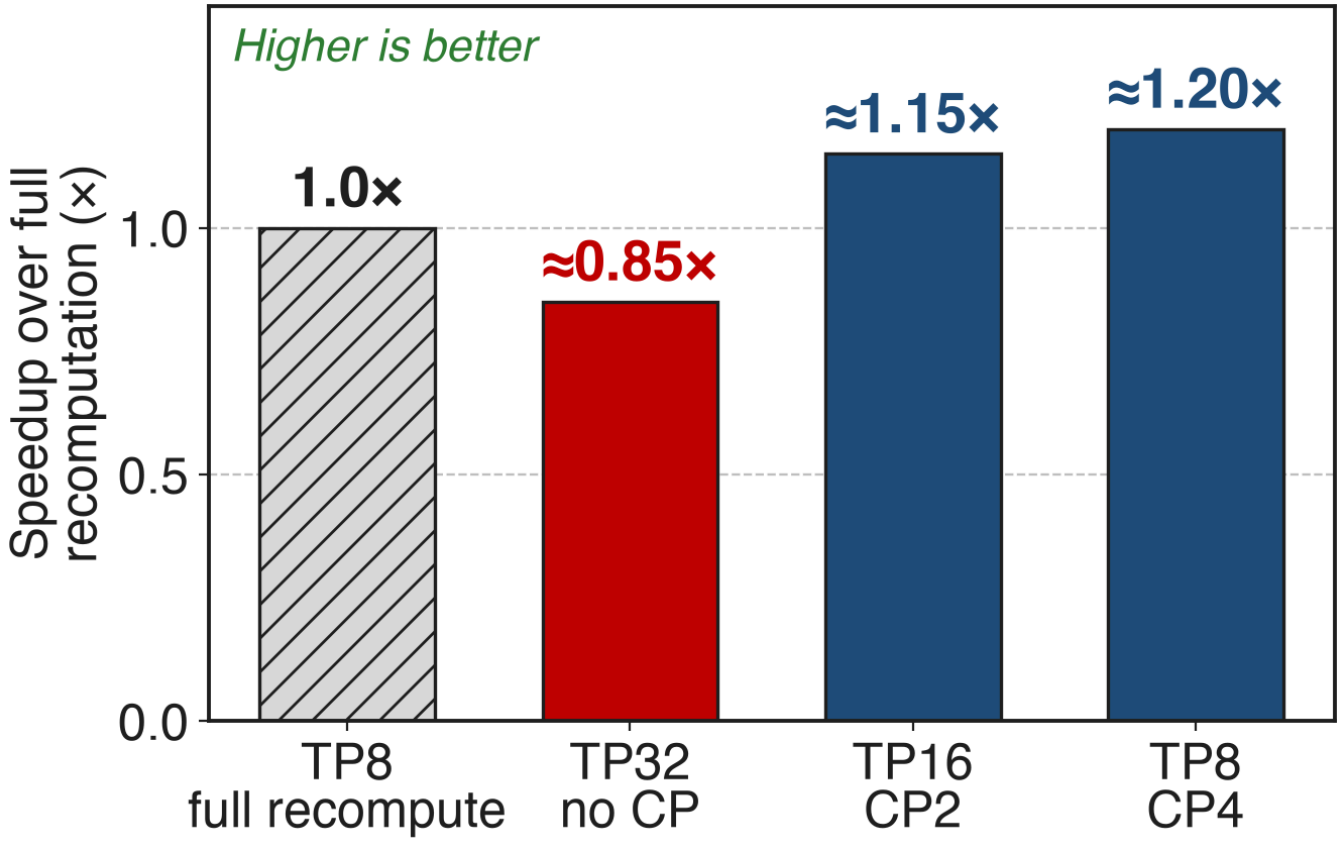
❸ Hierarchical CP matches comm to topology

- p2p (default): ring, overlaps attention
- a2a: heads instead of tokens, like Ulysses
- a2a+p2p: a2a in NVLink, ring across nodes



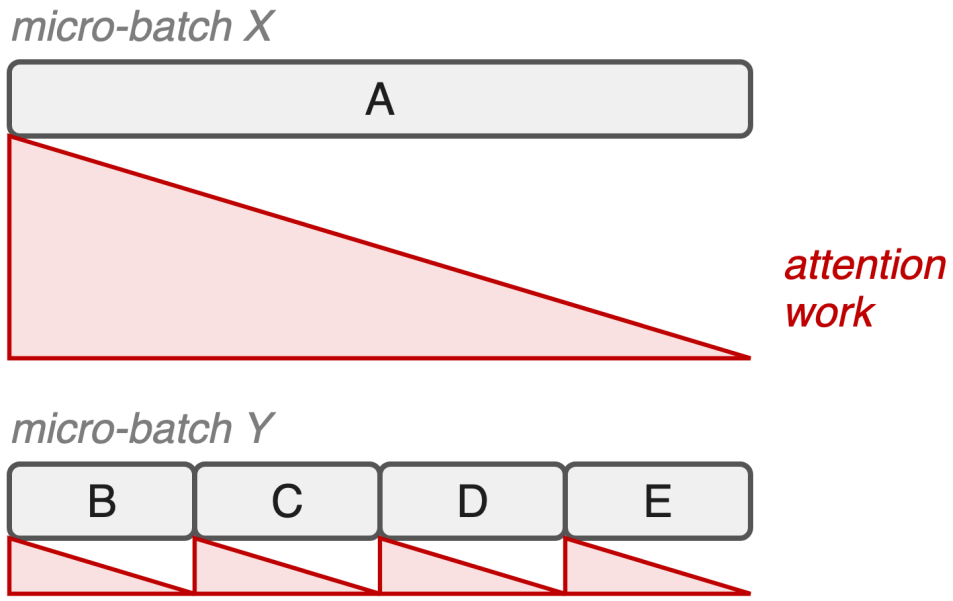
At 32K, TP8 + CP4 runs ≈1.2× faster

- 175B GPT, BF16, MHA
- FP8 + GQA: **≈1.3×** (TP8 CP4)
- At 16K: TP8 CP2 **≈1.25×**

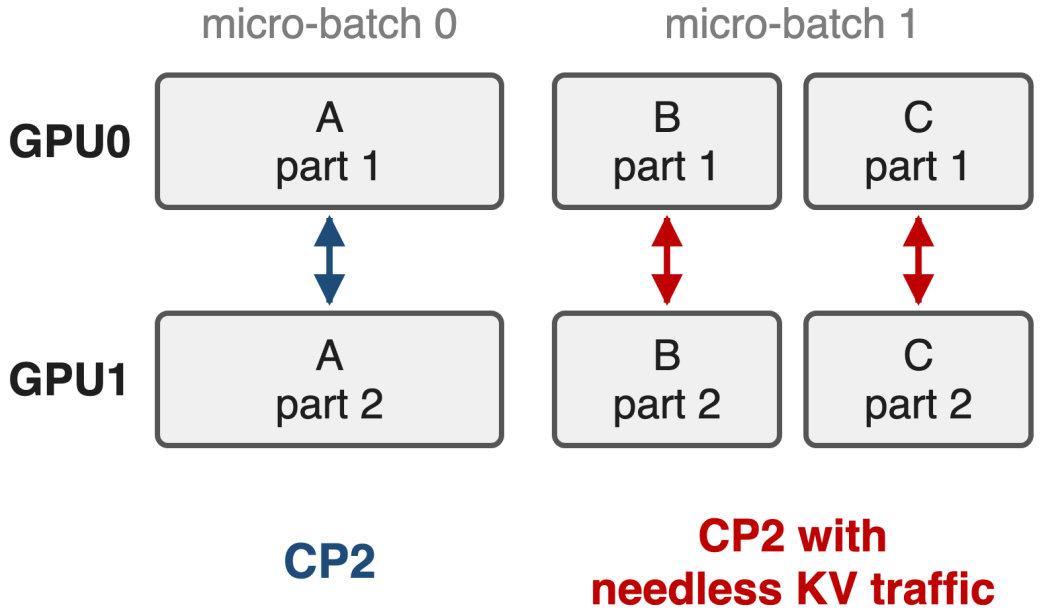


Packed variable-length batches break static CP

(a) Equal tokens but unequal work

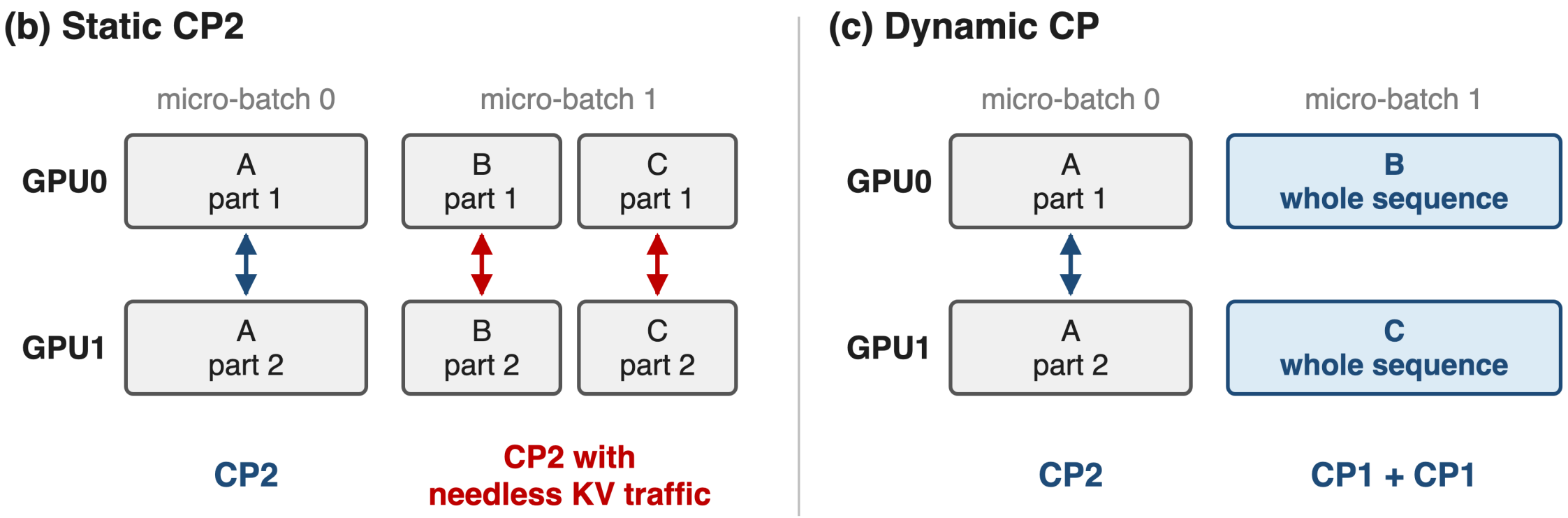


(b) Static CP2



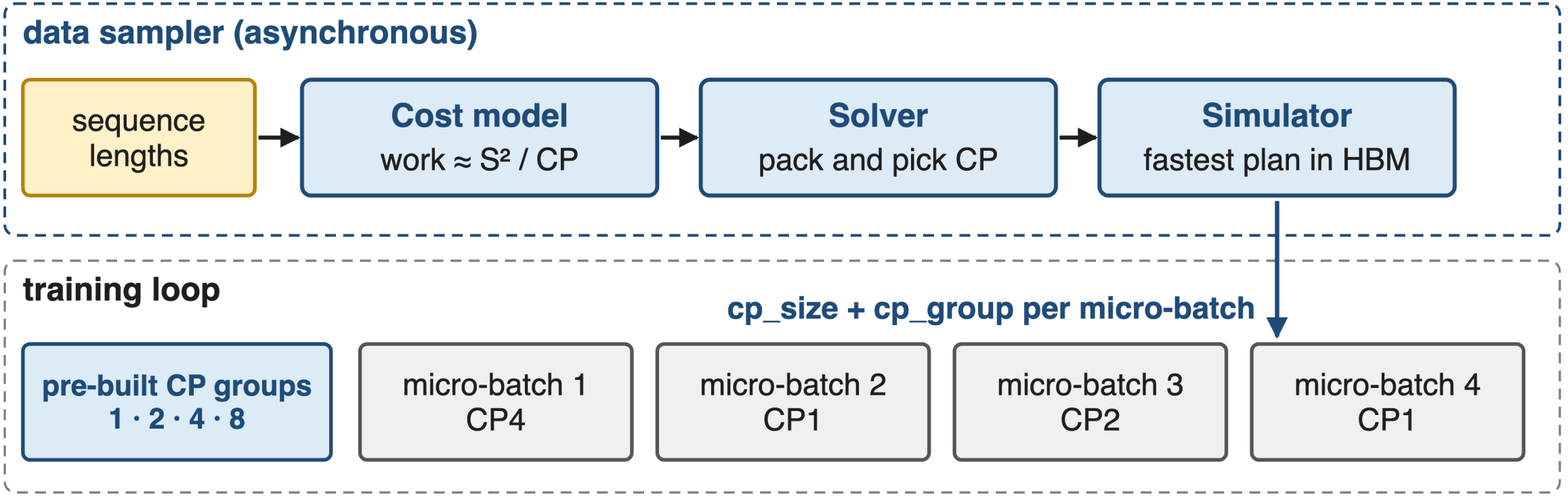
One CP size fits the longest sequence; short ones pay its KV traffic and DP ranks wait on the heaviest pack

❹ Dynamic CP picks the CP size per micro-batch



Resizing CP moves no weights or optimizer state: only token slices and the CP group change

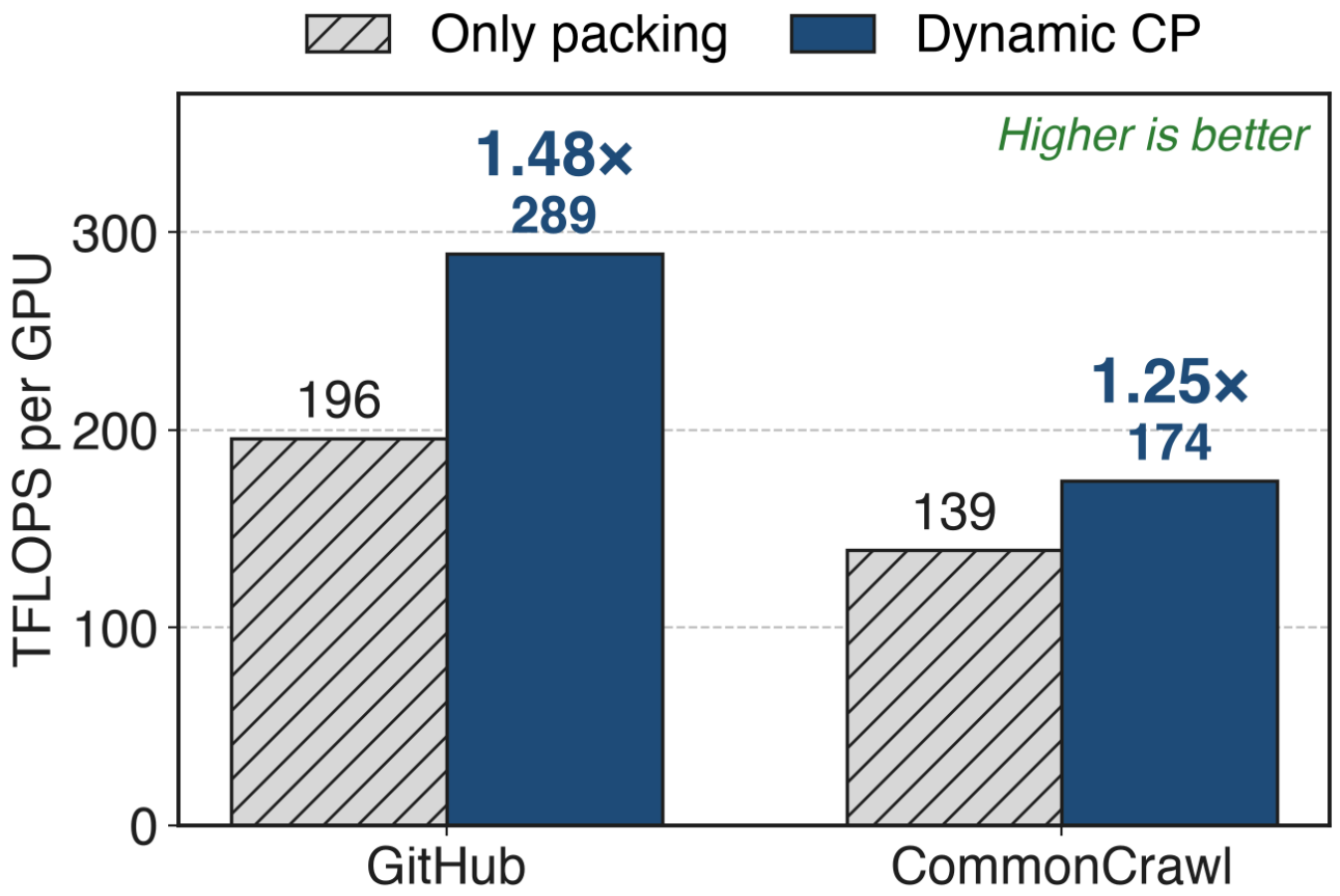
An async scheduler plans every micro-batch



CP groups for every power of two up to DP × CP are built once at start-up; PackedSeqParams carries the choice

Dynamic CP lifts throughput up to 1.48×

- Llama 13B, GBS 2048, PP8, CP8
- **> 35%** end to end at multi-thousand-GPU scale
- **35–60%** on highly imbalanced data



The cost: comm must hide under attention

- **Static CP**
- **Exposed ring** when shards are short or span nodes
- Weights not sharded: pair with TP or a sharded optimizer

- **Dynamic CP**
- Power-of-2 groups, **no CUDA Graphs**
- FlashAttention preferred; no MLA yet; no FSDP + PP

Summary

- Full recomputation for long context costs **~30%**
- **Insight**: only attention mixes tokens; split the sequence, stream KV in a ring
- Zigzag chunks, ring overlap, hierarchical a2a+p2p: **≈1.2–1.3×** over recompute
- Dynamic CP per micro-batch: **1.48×** on packed data

https://github.com/NVIDIA/Megatron-LM